

DEAP Pseudo Code

1. Import necessary libraries:

- import random
- from deap import base, creator, tools, algorithm
- Import default python libraries for solving

2. Define Fitness and Individual Types:

- Create a Fitness class (minimization or maximization):
creator.create("FitnessMulti", base.Fitness, weights=(-1.0, -1.0))
or for maximization: creator.create("FitnessMax", base.Fitness, weights=(-1.0, -1.0))
 - o Minimize reflection and minimize thickness
- Create an Individual class that inherits from list and has a fitness attribute:
creator.create("Individual", list, fitness=creator.FitnessMin)

3. Define Initialization, Evaluation, Crossover, and Mutation Functions:

- **Initialization**:
Define how individuals are created (e.g., random integers, binary strings):
function `create\_individual()` return a list of random values (e.g., random integers).
 - How are individuals created in our problem? (Random)

THICKNESS R CONTINUOUS AND MATERIAL ARE INTEGERS IN MATLAB MATLAB ROUNDS, DESIGNATE VALUES ARE INTEGERS INSTEAD?

SEE HOW IT EFFECTS CROSSOVER

Matlab has string (3 layer film, have chromosome thats 6 layers long (mat thickness and mat choice), when doing the cross over it picks a point (say pos 2) and mixes

Do cross over ONLY between integers and continuous variables

- **Evaluation**:
Define the fitness function to evaluate an individual (e.g., sum of squares):
function `evaluate(individual)` return the fitness value.
 - Is our fitness function the same as MatLab? (Minimize Reflection)
- **Crossover**:
Define how to recombine two parents (e.g., one-point or two-point crossover):
function `cx\_two\_point(ind1, ind2)` performs the crossover on two individuals.
 - How do we define crossover (MatLab default was scatter)
- **Mutation**:
Define how to mutate an individual (e.g., flip a random bit or change a random value):
function `mut\_flipbit(individual)` modify an individual's genes randomly.
 - MatLab used a bit-flip mutation; should we use the same? (Bit-flip is an adaptive mutation rate based off of performance usually ranges for 0.01-0.02)

4. Set Up the Evolutionary Algorithm Components:

- Create a Toolbox object to register the initialization, evaluation, selection, crossover, and mutation functions:

```
toolbox.register("individual", tools.initliterate, creator.Individual, create_individual)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
toolbox.register("evaluate", evaluate)
toolbox.register("mate", cx_two_point)
toolbox.register("mutate", mut_flipbit)
toolbox.register("select", tools.selTournament, tournsize=3)
```

- Create the initial population (e.g., 100 individuals):

```
population = toolbox.population(n=100)
```

- Using same tournament size (2), initial population (20) (Set for testing purposes)

5. Define the Algorithm Parameters:

- Set the number of generations to run the algorithm.
- Define probabilities for crossover and mutation (e.g., 70% crossover, 20% mutation).
 - Probability of crossover taken from MatLab (0.20~ will continue to change); Probability of mutation not defined in MatLab- what should we define as? Standard is between .01-0.05
 - MatLab set Elite Count (elite chromosomes) and they move onto the next gen

6. Evolutionary Loop:

- For each generation:
 - Evaluate the fitness of all individuals in the population.
 - Select a subset of the population for reproduction (e.g., tournament selection).
 - Clone the selected individuals to create offspring.
 - Apply crossover to the offspring with a certain probability.
 - Apply mutation to the offspring with a certain probability.
 - Evaluate the fitness of the mutated offspring.
 - Replace the old population with the new generation (offspring).

7. Output the Best Solution:

- After the evolutionary loop finishes, select and print the best individual from the population.